

Sicurezza

CdL in Informatica – L31

Crittografia Simmetrica

Prof. Emiliano Casalicchio

Department of Computer Science

Sapienza University of Rome



SAPIENZA
UNIVERSITÀ DI ROMA



Chapter 20 (CS)

Symmetric Encryption and Message Confidentiality



Symmetric Encryption

- Also referred to as:
 - Conventional encryption
 - Secret-key or single-key encryption
- Has five ingredients:
 - **Plaintext:** the original message or data fed into the algorithm
 - **Encryption algorithm** performs various substitutions and transformations on the plaintext.
 - **Secret key** is also input to the algorithm. The exact substitutions and transformations performed by the algorithm depend on the key
 - **Ciphertext:** the scrambled message produced as output. It depends on the plaintext and the secret key. For a given message, two different keys will produce two different ciphertexts
 - **Decryption algorithm:** the encryption algorithm run in reverse. It takes the ciphertext and the same secret key and produces the original plaintext.



Cryptographic systems

Mechanisms using algorithms, keys, and protocols to protect data confidentiality, integrity, authentication, and non-repudiation

Classified along three independent dimensions

The type of operations used for transforming plaintext to ciphertext

- Substitution
- Transposition

The number of keys used

- Symmetric
- Asymmetric

The way in which the plaintext is processed

- Block
- Stream

Type of operations used for transforming plaintext to ciphertext

All encryption algorithms are based on two general principles:

- Substitution
- Transposition

The fundamental requirement

- No information be lost (i.e., that all operations be reversible).

Substitution

- each element in the plaintext (bit, letter, group of bits or letters) is mapped into another element

Transposition

- Elements in the plaintext are rearranged.

Most systems
involve multiple
stages of
substitutions
and
transpositions



Number of keys used

Sender and receiver use the same key

- the system is referred to as symmetric, single-key, secret-key, or conventional encryption.

Sender and receiver each use a different key,

- the system is referred to as asymmetric, two-key, or public-key encryption.



The way in which the plaintext is processed

- **Block cipher**

- processes the input one block of elements at a time, producing an output block for each input block.

- **Stream cipher**

- processes the input elements continuously, producing output one element at a time



Block Cipher Structure

Symmetric block cipher consists of a **sequence of rounds** with **substitutions and permutations controlled by key**



Feistel Cipher Structure

16 rounds

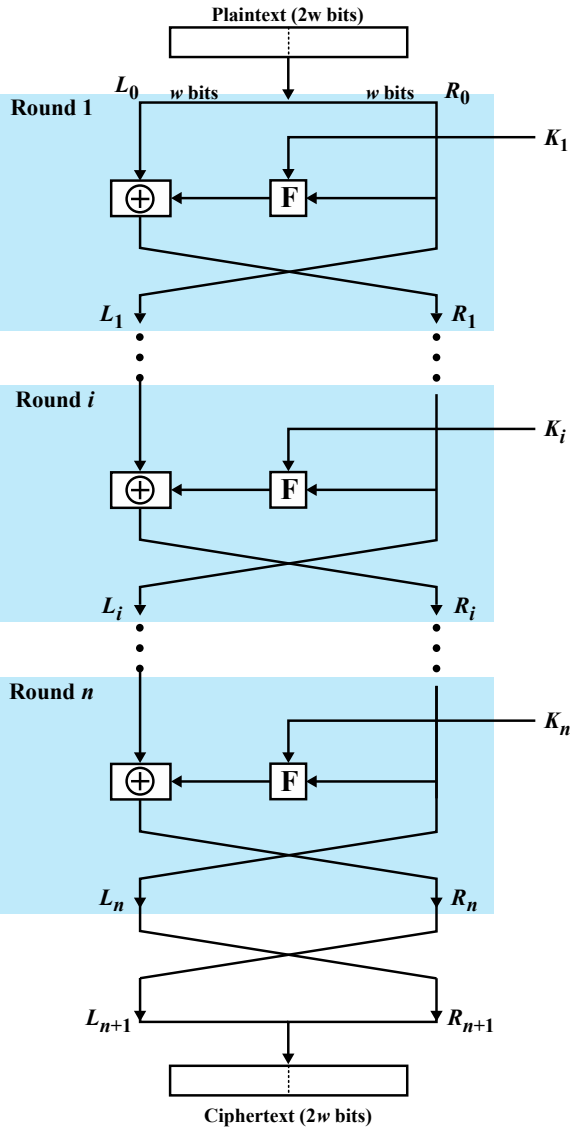
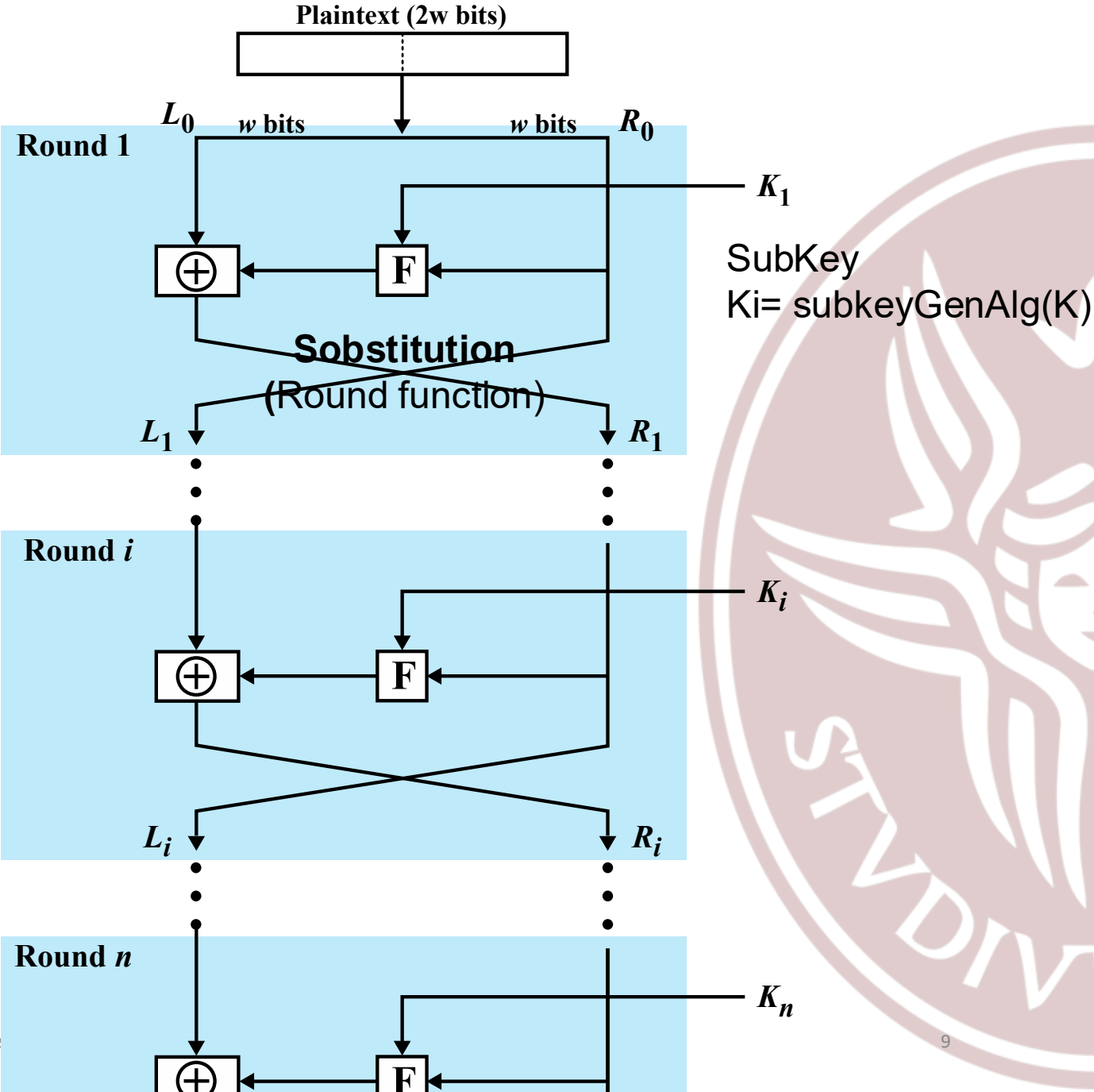


Figure 20.1 Classical Feistel Network



Block Cipher Structure: Parameters and design features

• :

Block size

Key size

Number of rounds

Subkey generation algorithm

Round function (substitution)

Fast software encryption/decryption

Ease of analysis



Symmetric block cipher parameters and design features

- Block size:
 - **Larger block sizes mean greater security** (all other things being equal) but reduced encryption/decryption speed.
 - A block size of **128 bits** is a **reasonable tradeoff** and is nearly universal among recent block cipher designs.
- Key size
 - **Larger key size means greater security** but may decrease encryption/ decryption speed.
 - Most common key length in modern algorithms is **128 - 256 bits**.
- Number of rounds
 - A single round offers inadequate security but that multiple rounds offer increasing security
 - Typical size is 16 rounds



Symmetric block cipher parameters and design features (cont'd)

- Subkey generation algorithm
 - Greater complexity in this algorithm should lead to greater difficulty of cryptanalysis.
- Round function
 - Greater complexity generally means greater resistance to cryptanalysis.
- Fast software encryption/decryption
 - encryption is embedded in applications or utility functions in such a way as to preclude a hardware implementation
 - The speed of execution of the algorithm becomes a concern
- Ease of analysis
 - As difficult as possible to cryptanalyze but easy to analyze
 - Easier to analyze for cryptanalytic vulnerabilities
 - DES, for example, does not have an easily analyzed functionality

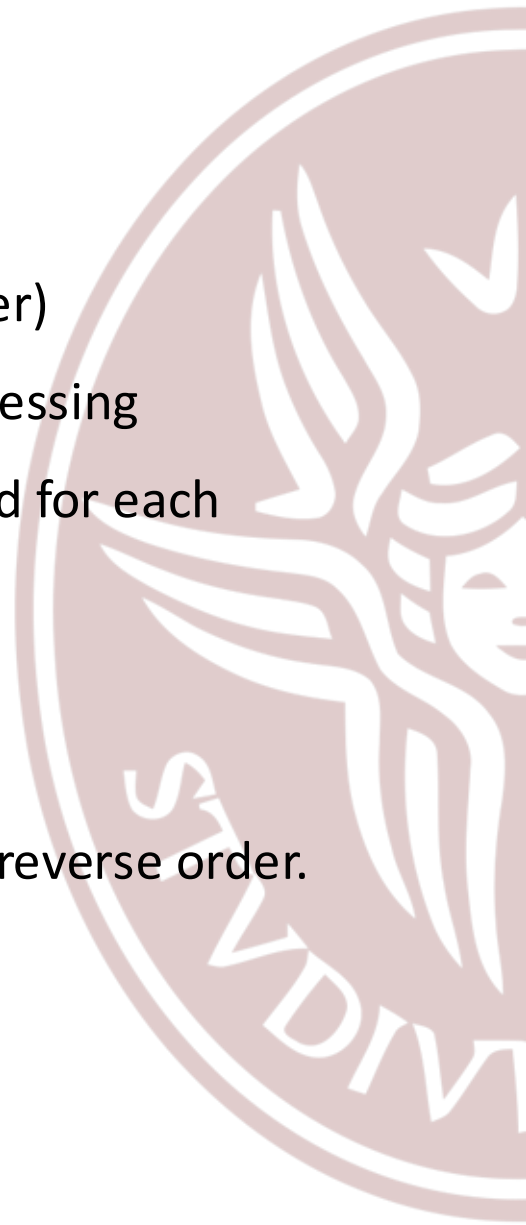
Cryptanalysis

The process of attempting to discover the plaintext or key



Data Encryption Standard (DES) – 1977 - FIPS PUB 46

- Algorithm is referred to as the Data Encryption Algorithm (DEA)
- Encryption process
 - The plaintext is 64 bits in length and the key is 56 bits in length (Block cipher)
 - DES structure is a minor variation of the Feistel network; 16 rounds of processing
 - From the original 56-bit key, 16 subkeys are generated, one of which is used for each round
- Decryption process
 - Essentially the same as the encryption process.
 - Use the ciphertext as input to the DES algorithm, but use the subkeys K_i in reverse order. That is, use K_{16} on the first iteration, K_{15} on the second iteration, and so on



3DES (1985)

- Uses three keys and three executions of the DES algorithm
 - $C = E(K_3, D(K_2, E(K_1, P)))$
- Decryption is the same with reverse key
 - $P = D(K_1, E(K_2, D(K_3, C)))$
- No cryptographic significance to the use of decryption for the second stage of 3DES encryption.
 - It allows users of 3DES to decrypt data encrypted by users of the older single DES:
 - $C = E(K_1, D(K_1, E(K_1, P))) = E[K_1, P]$
 - $P = D(K_1, E(K_1, D(K_1, C))) = D[K_1, C]$
- Key length
 - With three distinct keys 168 bits
 - With two keys ($K_1 = K_3$) 112 bits

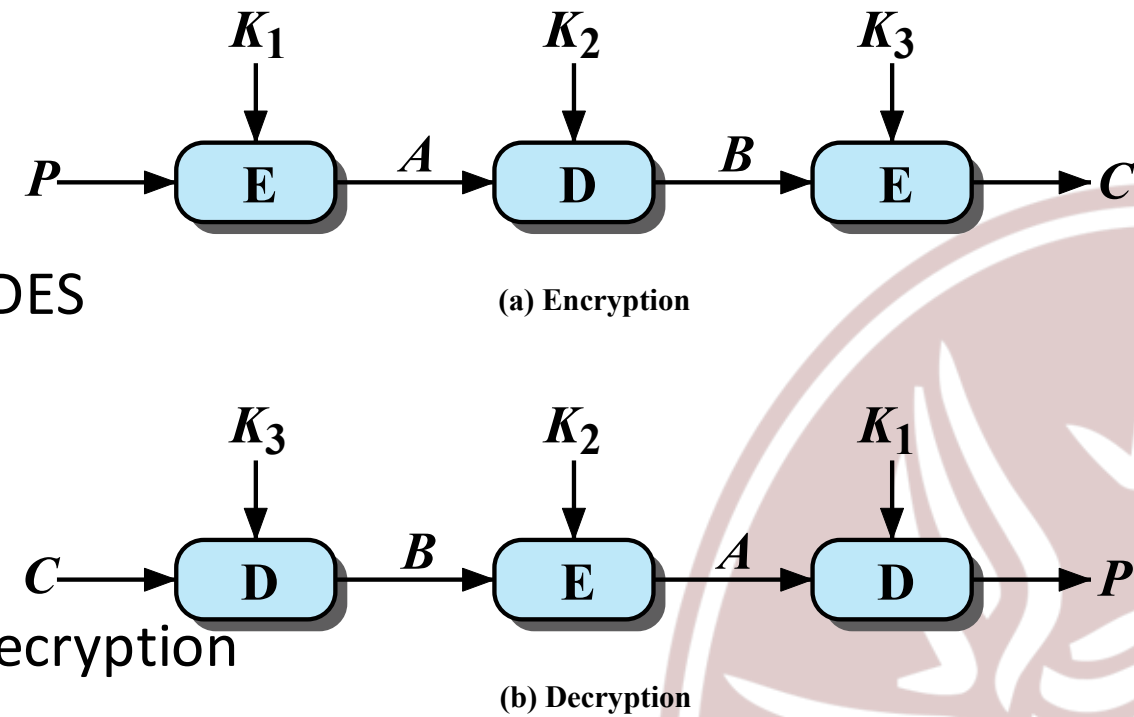
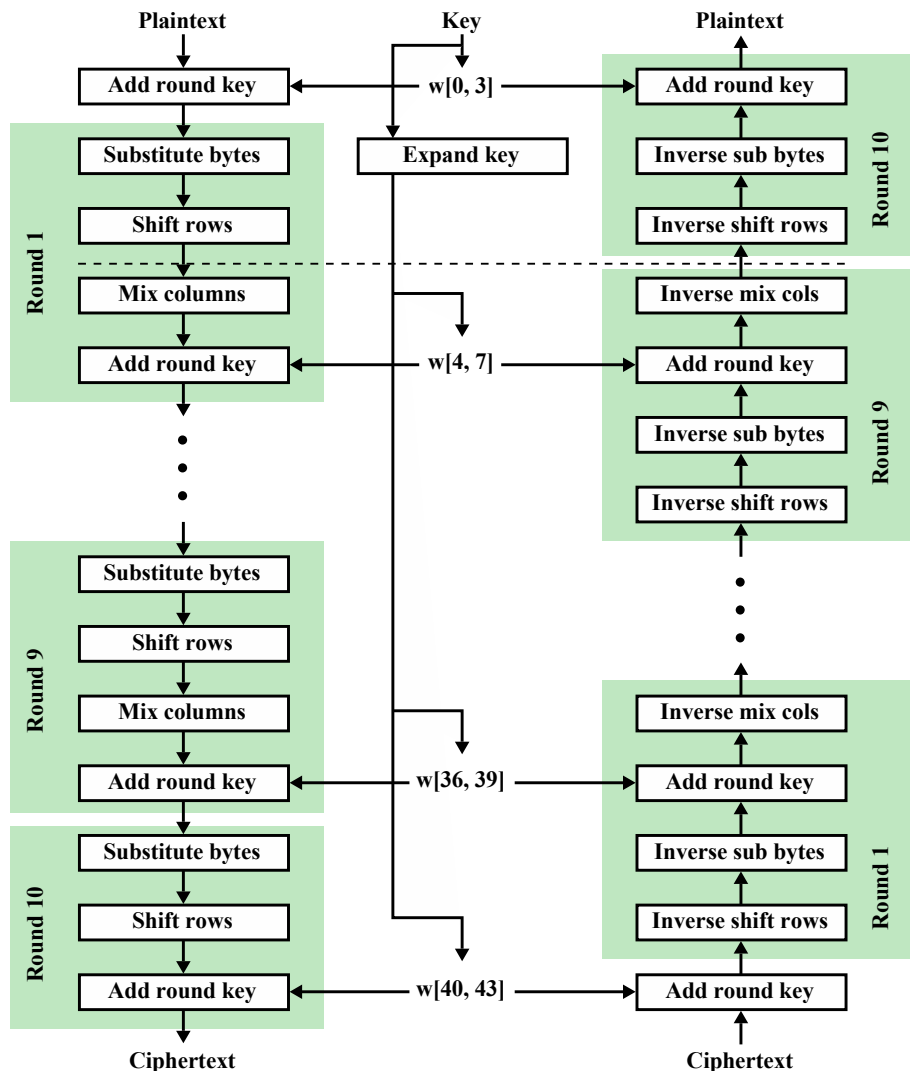


Figure 20.2 Triple DES

Advanced Encryption Standard (AES)

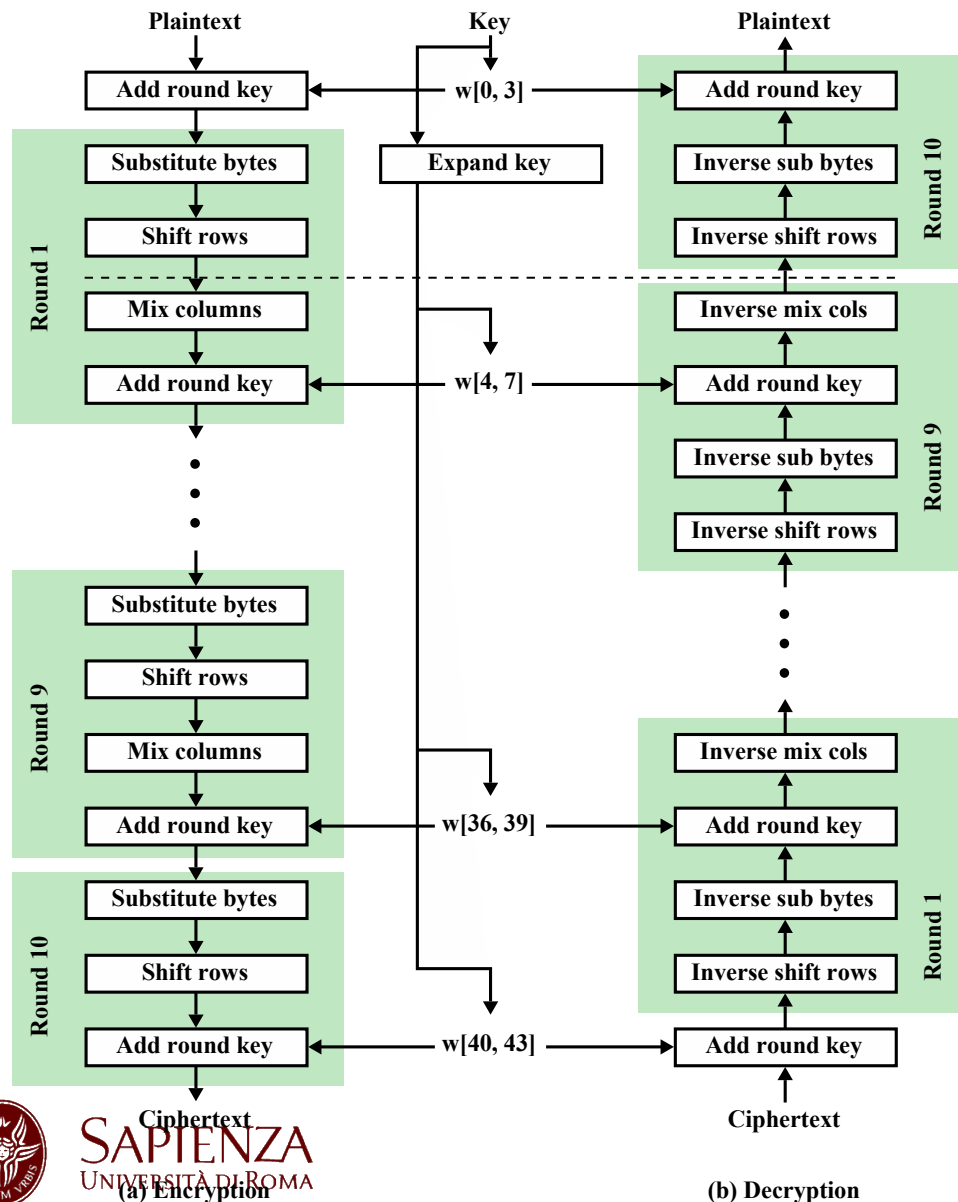


- **block length** of 128 bits and **key length** that can be **128**, 192, or 256 bits
- Input to the E/D algorithms is a single 128-bit block
 - this block is depicted as a square matrix of bytes (128bit = 16byte = 4x4matrix)
 - copied into the **State array**, which is modified at each stage of E/D
 - After the final stage, State is copied to an output matrix
- Key = 128bit depicted as a square matrix of bytes
 - expanded into an array of key schedule words
 - each word is 4 bytes and the total key schedule is 44 words for the 128-bit key
- The ordering of bytes within a matrix is by column both for input and key

B1	B5	B9	B13
B2	B6	B10	B14
B3	B7	B11	B15
B4	B8	B12	B16

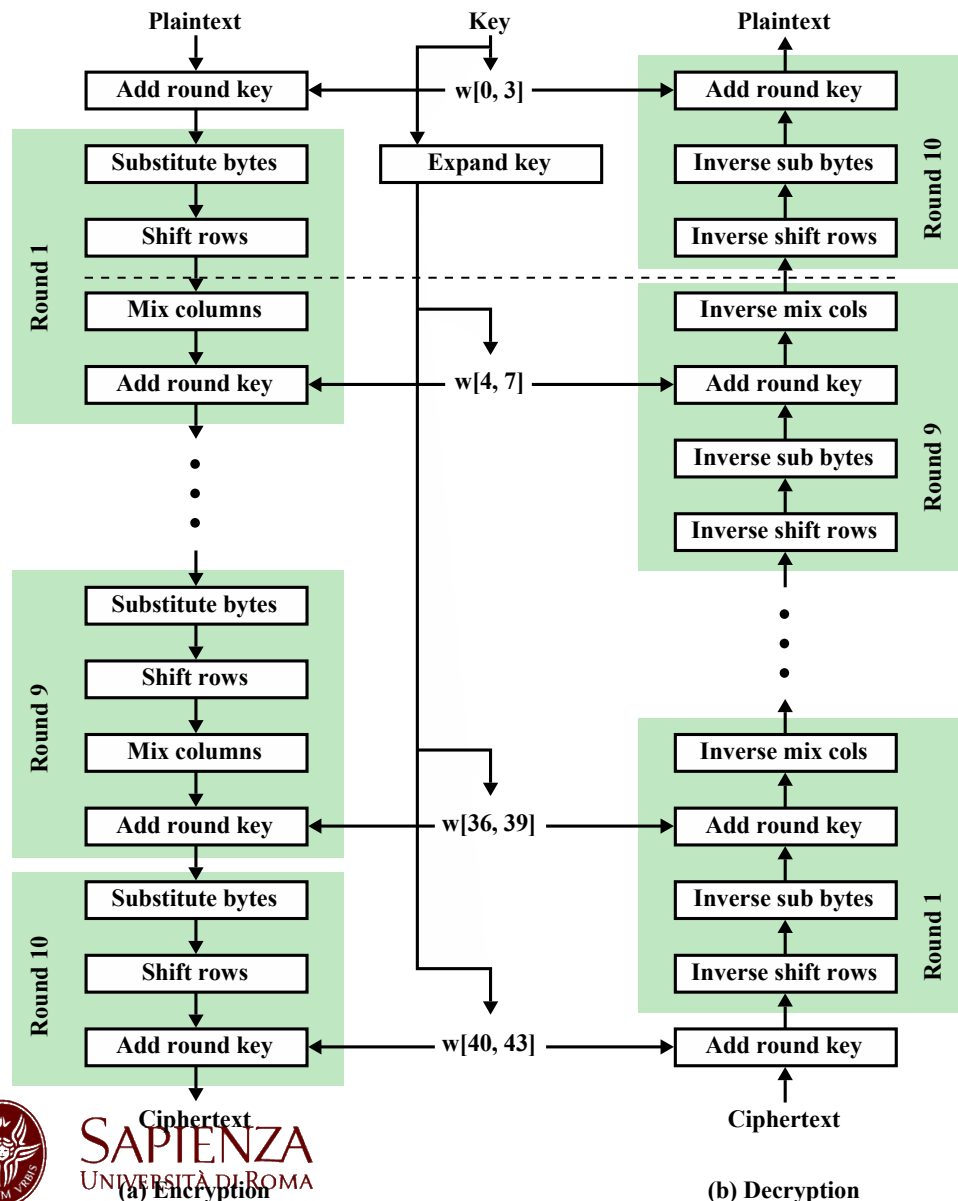


AES Insights



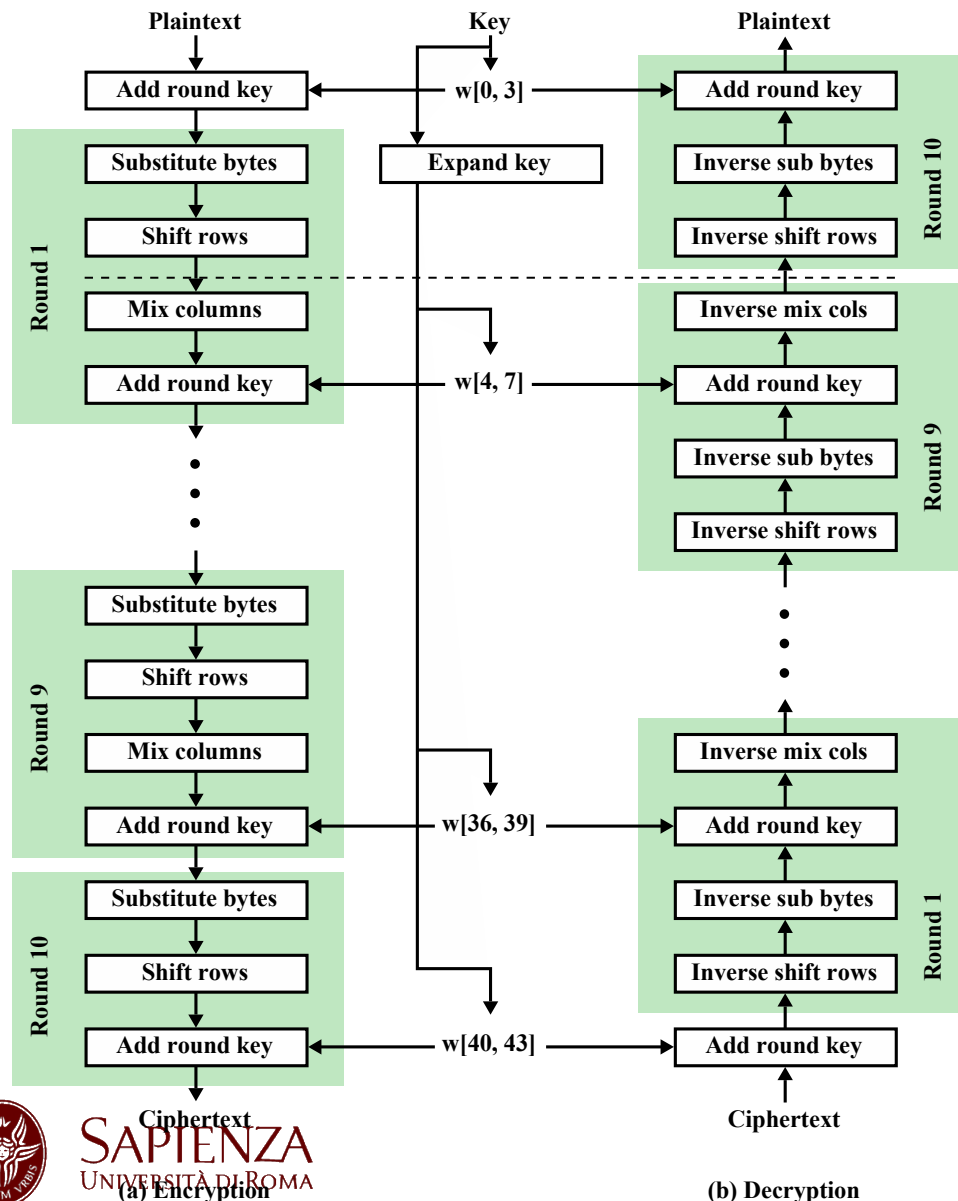
1. is not a Feistel structure but processes the entire data block in parallel during each round using substitutions and permutation
2. The key is expanded into an array of 44 32-bit words, $w[i]$. Four distinct words (128 bits) serve as a round key for each round.
3. Four different stages are used, one of permutation and three of substitution
 - **Substitute Bytes:** Uses a table, referred to as an S-box, to perform a byte-by-byte substitution of the block
 - **Shift Rows:** A simple permutation that is performed row by row
 - **Mix Columns:** A substitution that alters each byte in a column as a function of all the bytes in the column
 - **Add Round key:** A simple bitwise XOR of the current block with a portion of the expanded key

AES Insights (cont'd)



4. For both E/D, the cipher begins with an **Add Round Key** stage, followed by
 - nine rounds that each includes all four stages,
 - followed by a tenth round of three stages
5. Only the **Add Round Key** stage makes use of the key. For this reason,
 - the cipher begins and ends with an Add Round Key stage.
 - **Any other stage, applied at the beginning or end, is reversible without knowledge of the key and so would add no security**
6. The Add Round Key stage by itself does not provide strong security
 - The other three stages together scramble the bits, but by themselves would provide no security
 - We can view the cipher as alternating operations of XOR encryption (Add Round Key) of a block, followed by scrambling of the block followed by XOR encryption, and so on. This scheme is both efficient and highly secure

AES Insights (cont'd)



7. Each stage is easily reversible

- For the Substitute Byte, Shift Row, and Mix Columns stages, an inverse function is used in the decryption algorithm.
- For the Add Round Key stage, the inverse is achieved by XORing the same round key to the block, using the result that $A \oplus A \oplus B = B$

8. The decryption algorithm makes use of the expanded key in reverse order.

- The decryption algorithm is not identical to the encryption algorithm.
- This is a consequence of the particular structure of AES.

9. Once it is established that all four stages are reversible, it is easy to verify that decryption does recover the plaintext

10. The final round of both E/D consists of only three stages.

- this is a consequence of the particular structure of AES and is required to make the cipher reversible.

Details of the encryption round

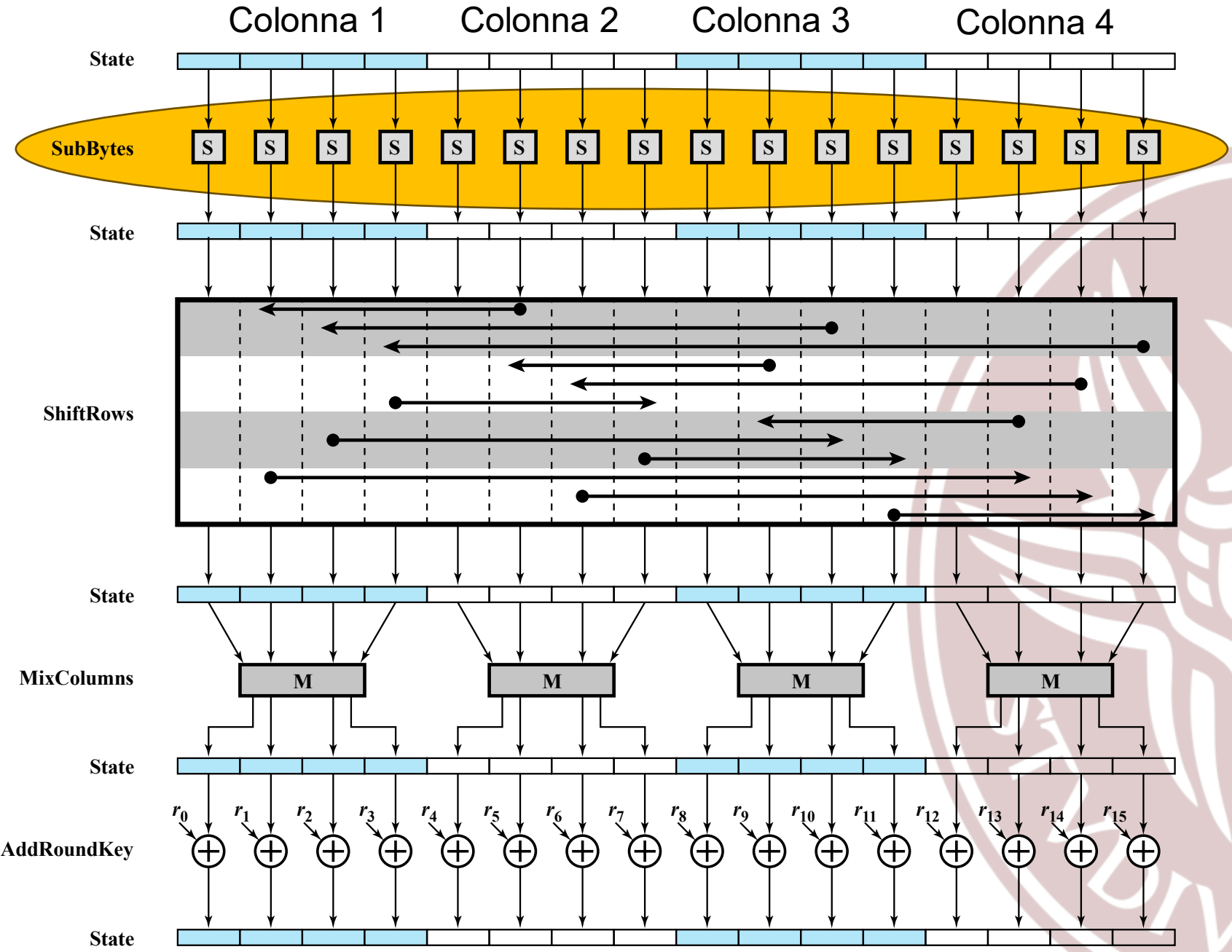


Figure 20.4 AES Encryption Round

Forward substitute byte transformation (SubByte)

Table 20.2 AES S-Boxes

(a) S-box

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

- simple table lookup
- AES defines a 16x16 matrix of byte values, called an S-box that contains a permutation of all possible 256 8-bit values
- Mapping:
 - The leftmost 4 bits of the byte are used as a row value and the rightmost 4 bits are used as a column value
 - These row and column values serve as indexes into the S-box to select a unique 8-bit output value
 - Example: {95} hex --> [row 9, col 5] = {2A}



Inverse substitute byte transformation: InvSubBytes

(b) Inverse S-box

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

- makes use of the inverse S-box
- Example:
 - {2A} --> {95}
- The S-box is designed to be resistant to known cryptanalytic attacks
- **The AES developers sought a design that has**
 - a low correlation between input bits and output bits
 - the property that the output cannot be described as a simple mathematical function of the input

Details of the encryption round

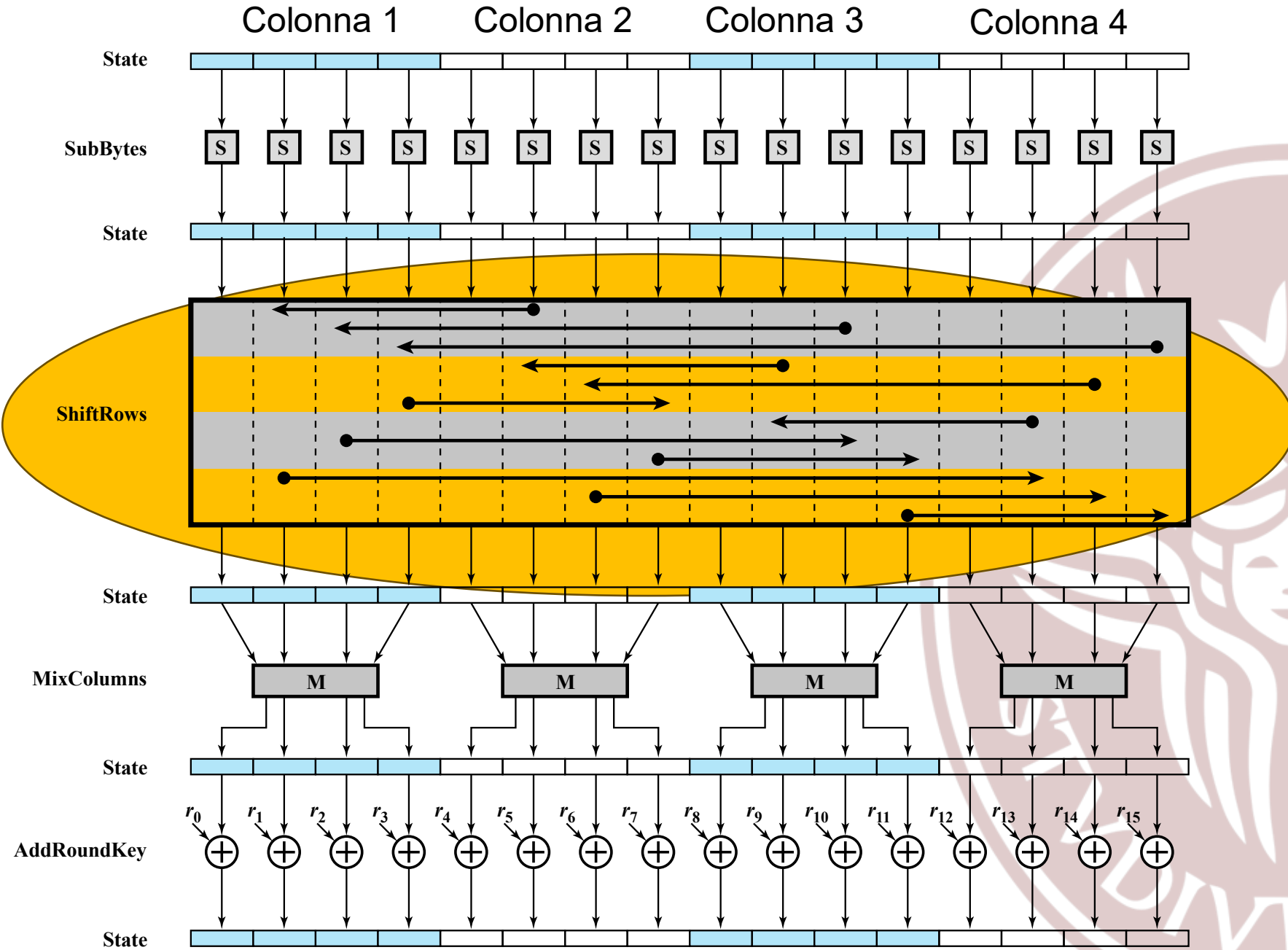


Figure 20.4 AES Encryption Round

Forward shift row transformation: ShiftRows

- Shift E=left; D=right
 - 1st row of State is not altered
 - 2nd row, a 1-byte circular left shift is performed.
 - 3rd row, a 2-byte circular left shift is performed.
 - 4th row, a 3-byte circular left shift is performed
- Substantial transformation
 - the round key is applied to State column by column.
 - a row shift moves an individual byte from one column to another, which is a linear distance of a multiple of 4 bytes.
 - the transformation ensures that **the four bytes of one column are spread out to four different columns.**

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

→

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

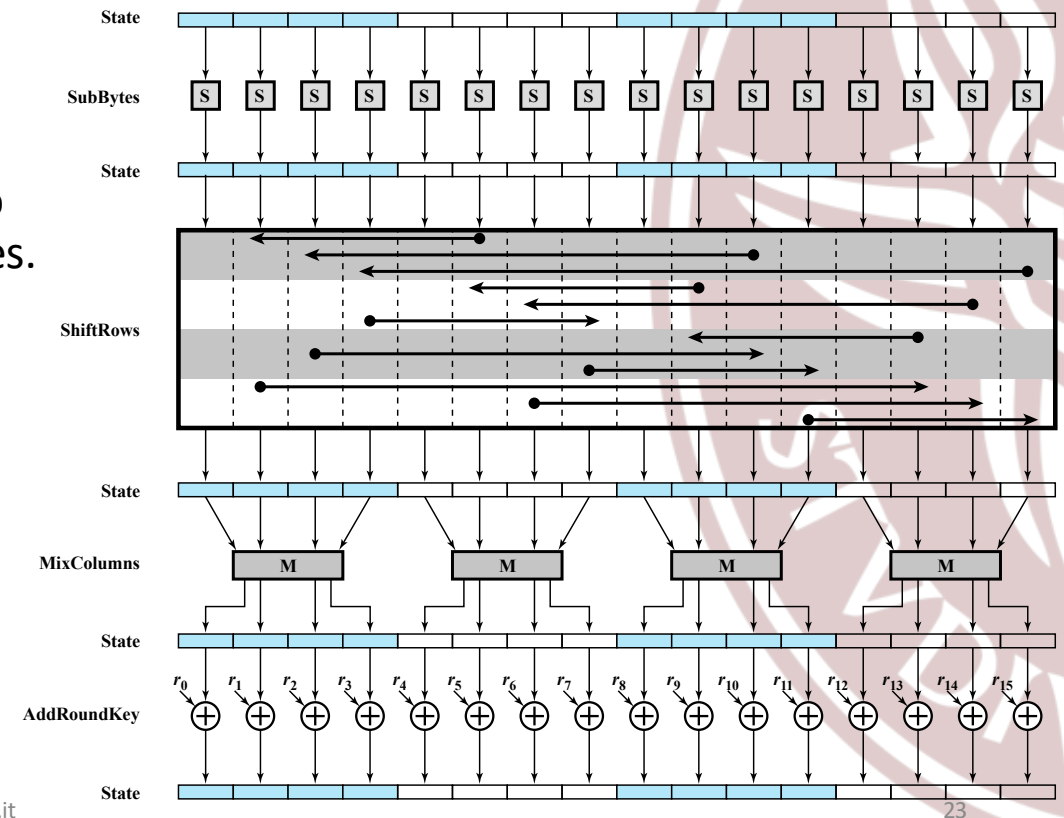


Figure 20.4 AES Encryption Round

Mix column transformation -MixColumns

- Each byte of a column is mapped into a new value that is a function of all four bytes in the column
- The mapping
 - makes use of equations over finite fields (not subject of the course).
 - is designed to provide a good mixing among the bytes of each column
- The mix column transformation combined with the shift row transformation ensures that after a few rounds, **all output bits depend on all input bits**

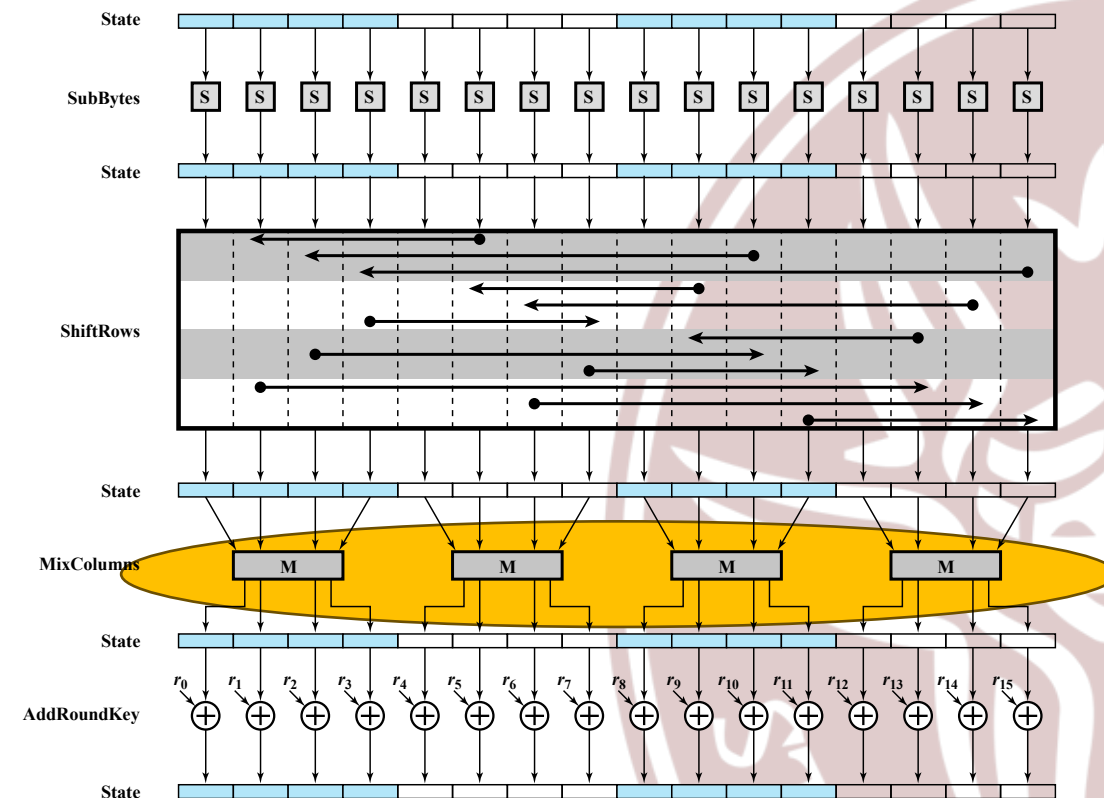


Figure 20.4 AES Encryption Round

Add round key transformation - AddRoundKey

- The 128 bits of State are bitwise XORed with the 128 bits of the round key
- The operation is viewed as a column-wise operation between the four bytes of a State column and one word of the round key
 - it can also be viewed as a byte-level operation.
- is as simple as possible and affects every bit of State.
 - The complexity of the round key expansion, plus the complexity of the other stages of AES, ensure security
- **The inverse** is identical to the forward add round key transformation, because the XOR operation is its own inverse.

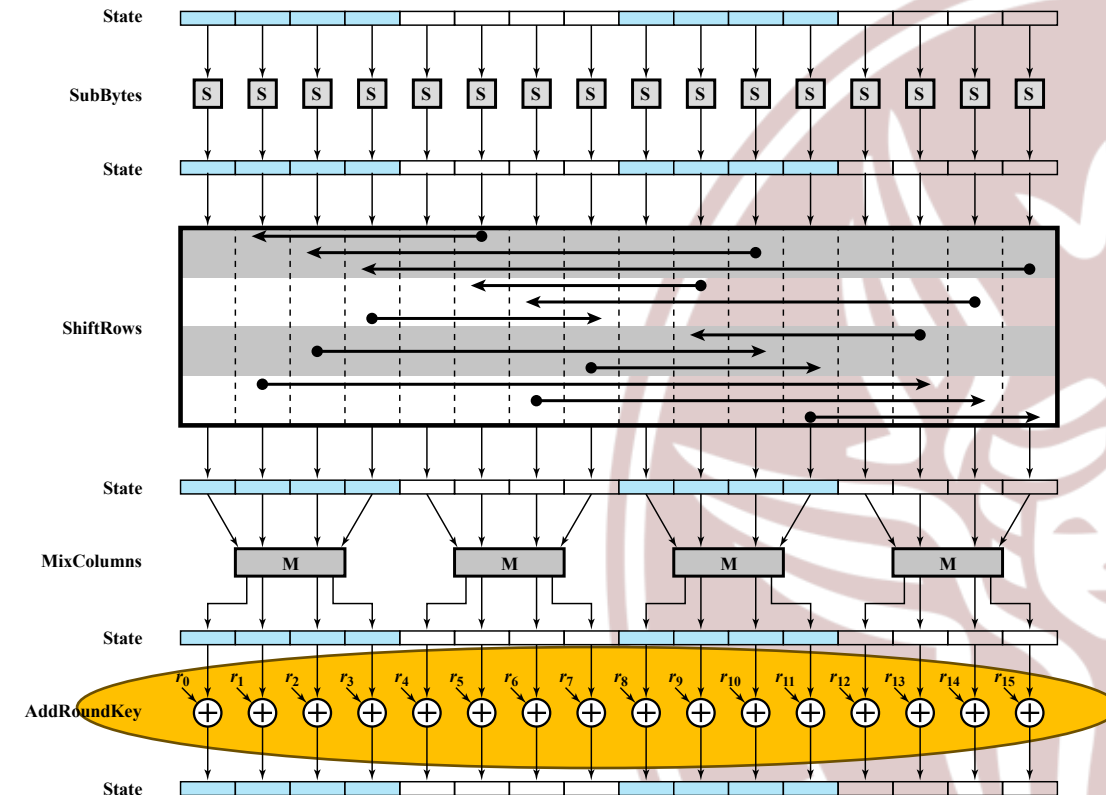


Figure 20.4 AES Encryption Round

AES key expansion algorithm

- Takes as input a 4-word (16-byte) key and produces a linear array of 44 words (176 bytes). This is sufficient to provide
 - 4-word round key for the initial Add Round Key stage
 - each of the 10 rounds of the cipher

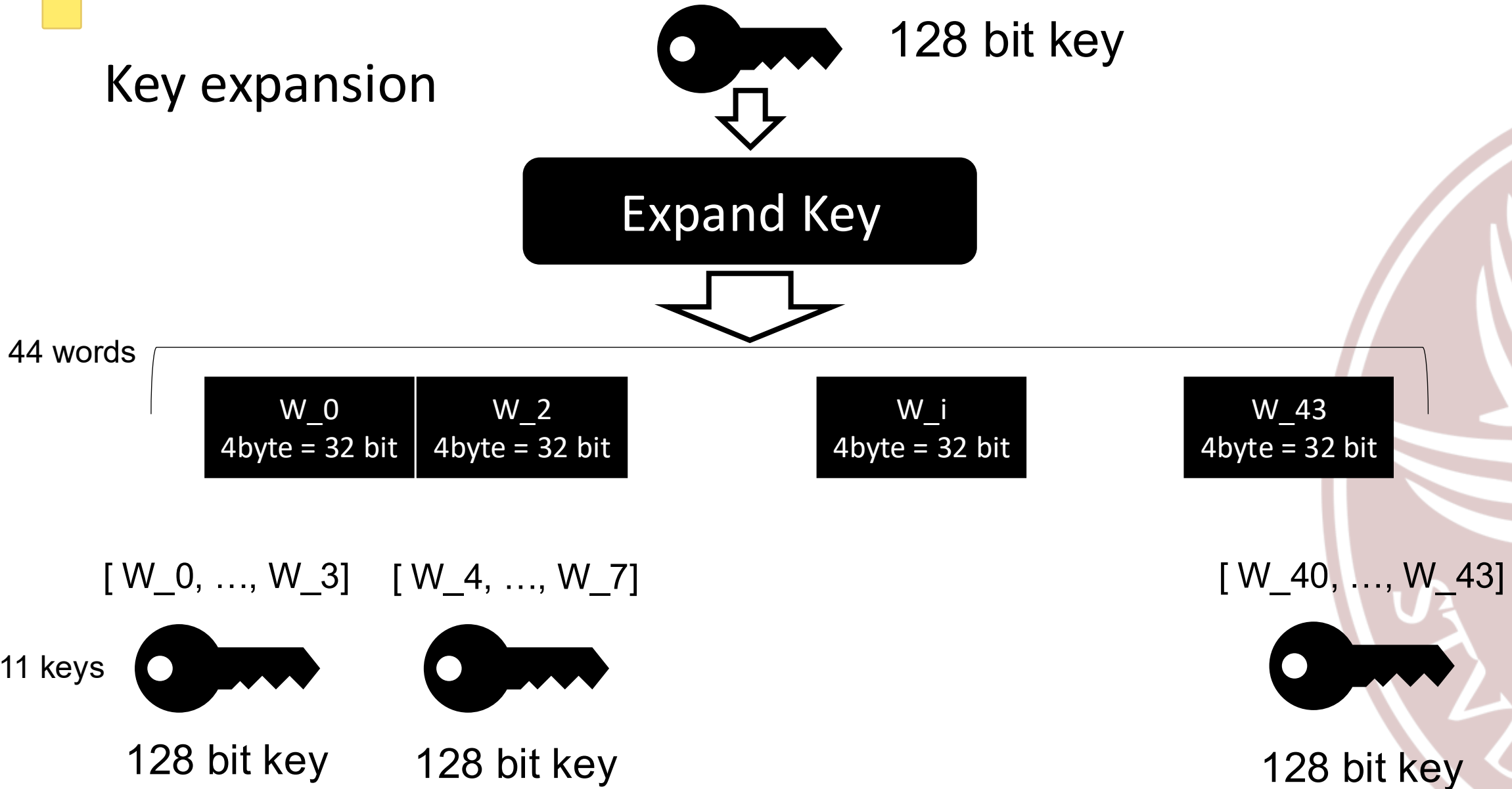
Steps:

1. The key is copied into the first four words of the expanded key.
2. The remainder of the expanded key is filled in four words at a time
 - Each added word $w[i]$ depends on
 - $w[i - 1]$, and $w[i - 4]$
 - A complex finite-field algorithm is used in generating the expanded key.





Key expansion



Stream Ciphers

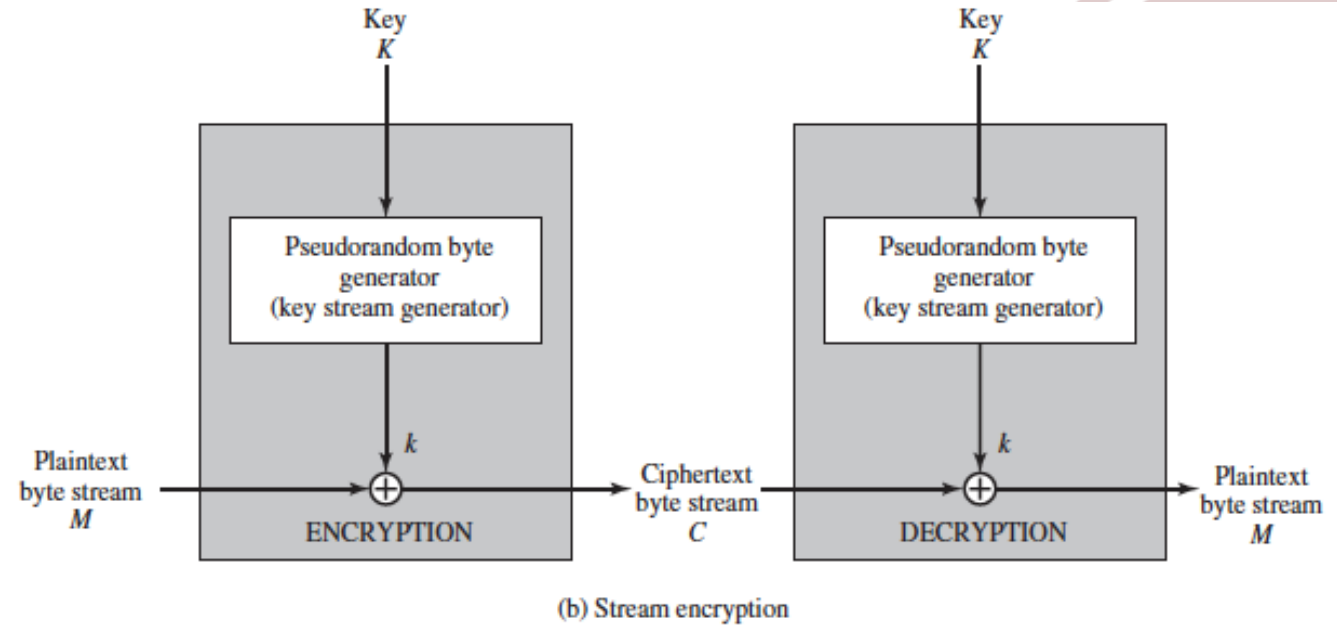
Processes input elements continuously (bit, byte, words, blocks)

Key input to a pseudorandom bit generator

Produces stream of random like numbers

Unpredictable without knowing input key

XOR keystream output with plaintext bytes



RC4 (1987 by Ron Rivest for RSA Security)

- Variable key-size stream cipher with byte-oriented operations
- Is based on the use of a random permutation
- Small code
 - Eight to sixteen machine operations are required per output byte
- Used in
 - SSL/TLS
 - WEP (Wired Equivalent Privacy)
 - WPA (WiFi Protected Access)
- Kept as a trade secret by RSA Security.
 - In September 1994, the RC4 algorithm was anonymously posted on the Cypherpunks anonymous remailers list

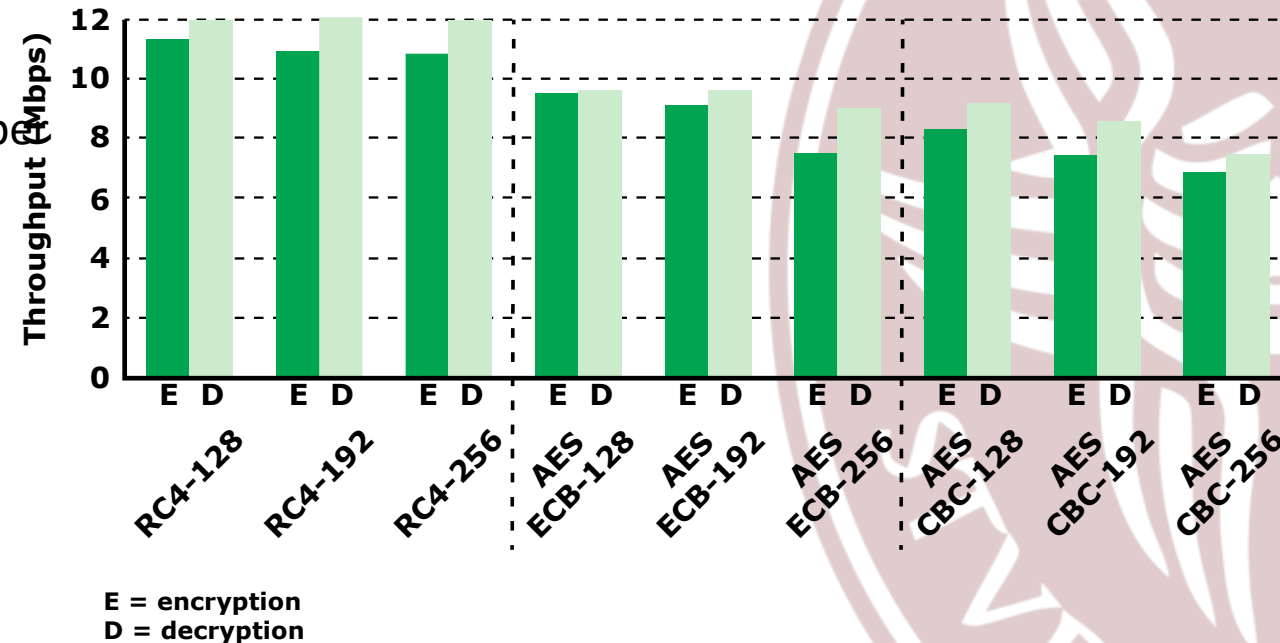
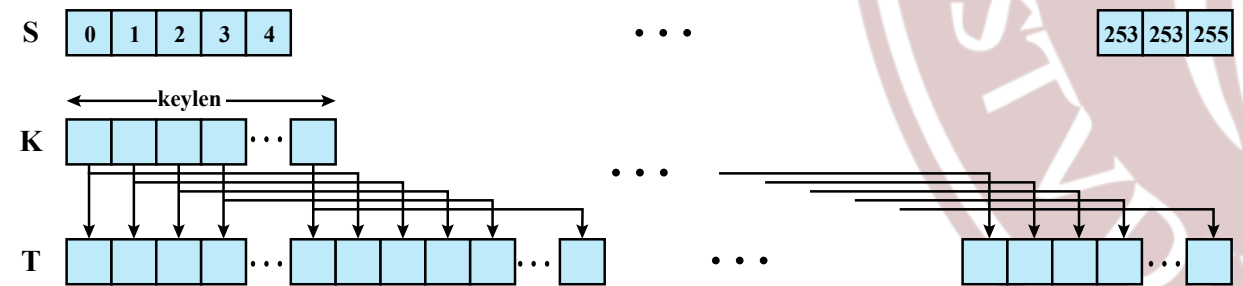


Figure 20.5 Performance Comparison of Symmetric Ciphers on a 3-GHz Processor

RC4: Initialization phase

- Variable-length key (1 to 256 bytes) used to initialize a 256-byte state vector S , with elements $S[0]$, $S[1]$, ..., $S[255]$
 - At all times, S contains a permutation of all 8-bit numbers from 0 through 255.
 - For encryption and decryption, a byte k is generated from S by selecting one of the 255 entries
 - As each value of k is generated, the entries in S are once again permuted
- If the length of the key K is 256 bytes, then K is transferred to T (Temporary vector). Otherwise, a key of length $keylen < 256$ bytes is copied in T as many times as necessary to fill out T

```
/* Initialization */  
for i = 0 to 255 do  
  S[i] = i;  
  T[i] = K[i mod keylen];
```

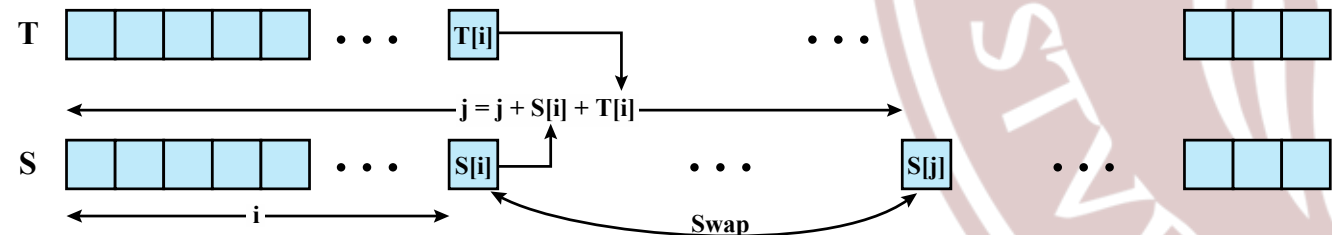


(a) Initial state of S and T

RC4: Initial Permutation of S

- We use T to produce the initial permutation of S. This involves starting with S[0] and going through to S[255], and, for each S[i], swapping S[i] with another byte in S according to a scheme dictated by T[i]
- Because the only operation on S is a swap, the only effect is a permutation.
 - S still contains all the numbers from 0 through 255
- **Once the S vector is initialized, the input key is no longer used**

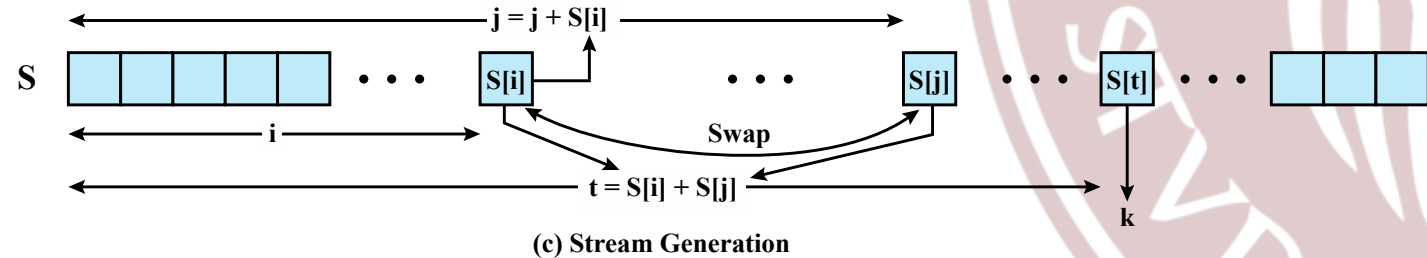
```
/* Initial Permutation of S */  
j = 0;  
for i = 0 to 255 do  
    j = (j + S[i] + T[i]) mod 256;  
    Swap (S[i], S[j]);
```



RC4: Stream Generation

- Involves cycling through all the elements of $S[i]$, and, for each $S[i]$, swapping $S[i]$ with another byte in S according to a scheme dictated by the current configuration of S .
 - After $S[255]$ is reached, the process continues, starting over again at $S[0]$:
- **To encrypt**, XOR the value k with the next byte of plaintext.
- **To decrypt**, XOR the value k with the next byte of ciphertext

```
/* Stream Generation */  
i, j = 0;  
while (true)  
    i = (i + 1) mod 256;  
    j = (j + S[i]) mod 256;  
    Swap (S[i], S[j]);  
    t = (S[i] + S[j]) mod 256;  
    k = S[t];
```



Block Cipher Modes of Operation

NIST SP 800-38A (Recommendation for Block Cipher Modes of Operation: Methods and Techniques, December 2001)

- Introduced to regulate the way a plaintext longer than the block size should be encrypted
- Apply to DES, 3DES, AES and any other block cipher algorithm

Mode	Description	Typical Application
Electronic Codebook (ECB)	Each block of 64 plaintext bits is encoded independently using the same key.	•Secure transmission of single values (e.g., an encryption key)
Cipher Block Chaining (CBC)	The input to the encryption algorithm is the XOR of the next 64 bits of plaintext and the preceding 64 bits of ciphertext.	•General-purpose block-oriented transmission •Authentication
Cipher Feedback (CFB)	Input is processed s bits at a time. Preceding ciphertext is used as input to the encryption algorithm to produce pseudorandom output, which is XORed with plaintext to produce next unit of ciphertext.	•General-purpose stream-oriented transmission •Authentication
Output Feedback (OFB)	Similar to CFB, except that the input to the encryption algorithm is the preceding DES output.	•Stream-oriented transmission over noisy channel (e.g., satellite communication)
Counter (CTR)	Each block of plaintext is XORed with an encrypted counter. The counter is incremented for each subsequent block.	•General-purpose block-oriented transmission •Useful for high-speed requirements



Electronic Codebook (ECB) mode

- Simplest mode, Plaintext is handled b bits at a time and each block is encrypted using the same key
- “Codebook” is used because there is a unique ciphertext for every b -bit block of plaintext
 - Not secure for long messages since repeated plaintext is seen in repeated ciphertext
 - For example, if it is known that the message always starts out with certain predefined field
 - If the message has repetitive elements, with a period of repetition a multiple of b bits, then these elements can be identified by the analyst
- To overcome security deficiencies you need a technique where the same plaintext block, if repeated, produces different ciphertext blocks



Cipher Block Chaining (CBC) mode

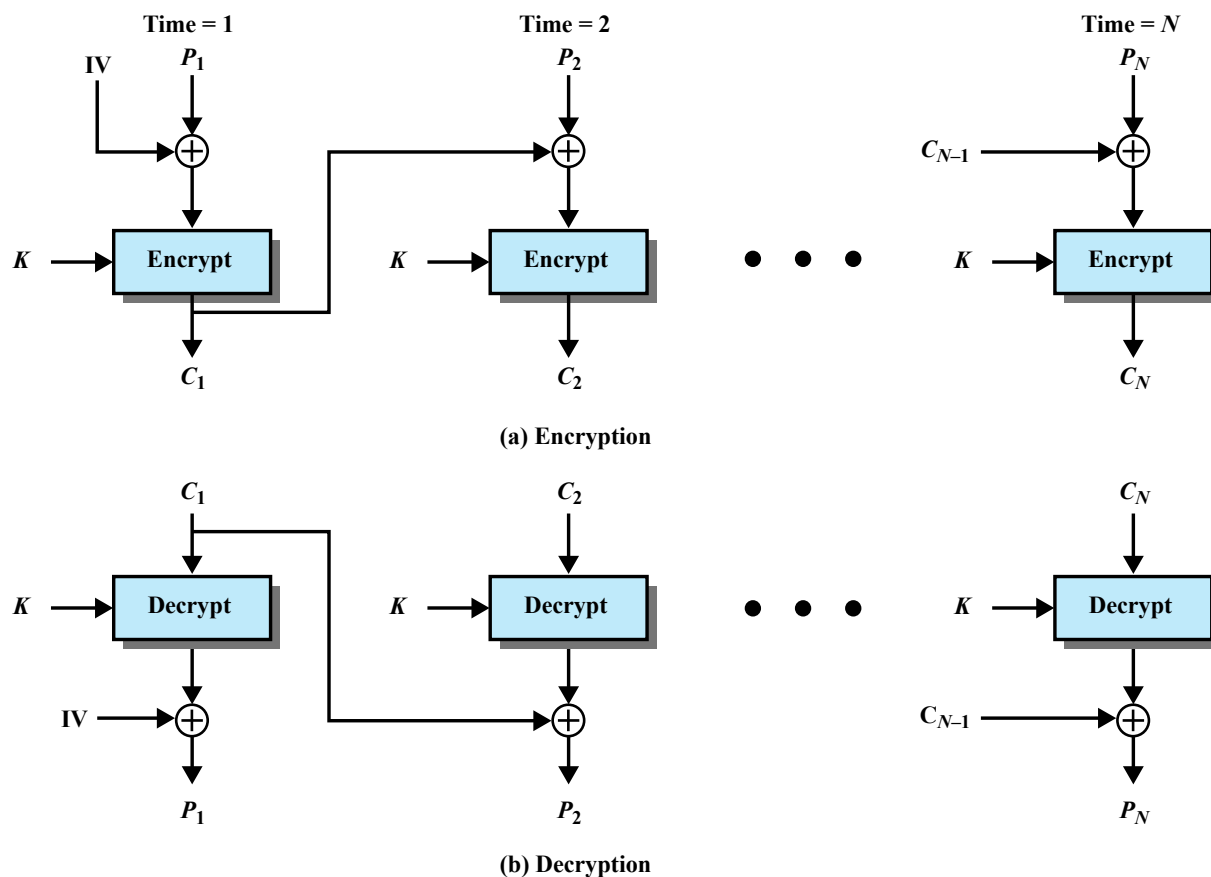


Figure 20.7 Cipher Block Chaining (CBC) Mode

Encryption

- the same key is used for each block, we need and Initialization vector (IV)
- we have chained together the processing of the sequence of plaintext blocks
- The input to the encryption function for each plaintext block bears no fixed relationship to the plaintext block.
- Repeating patterns of b bits are not exposed

Decryption

- Each cipher block is passed through the decryption algorithm
- The result is XORed with the preceding ciphertext block to produce the plaintext block

CBC Decryption (proof)

- For decryption, each cipher block is passed through the decryption algorithm. The result is XORed with the preceding ciphertext block to produce the plaintext block. To see that this works, we can write

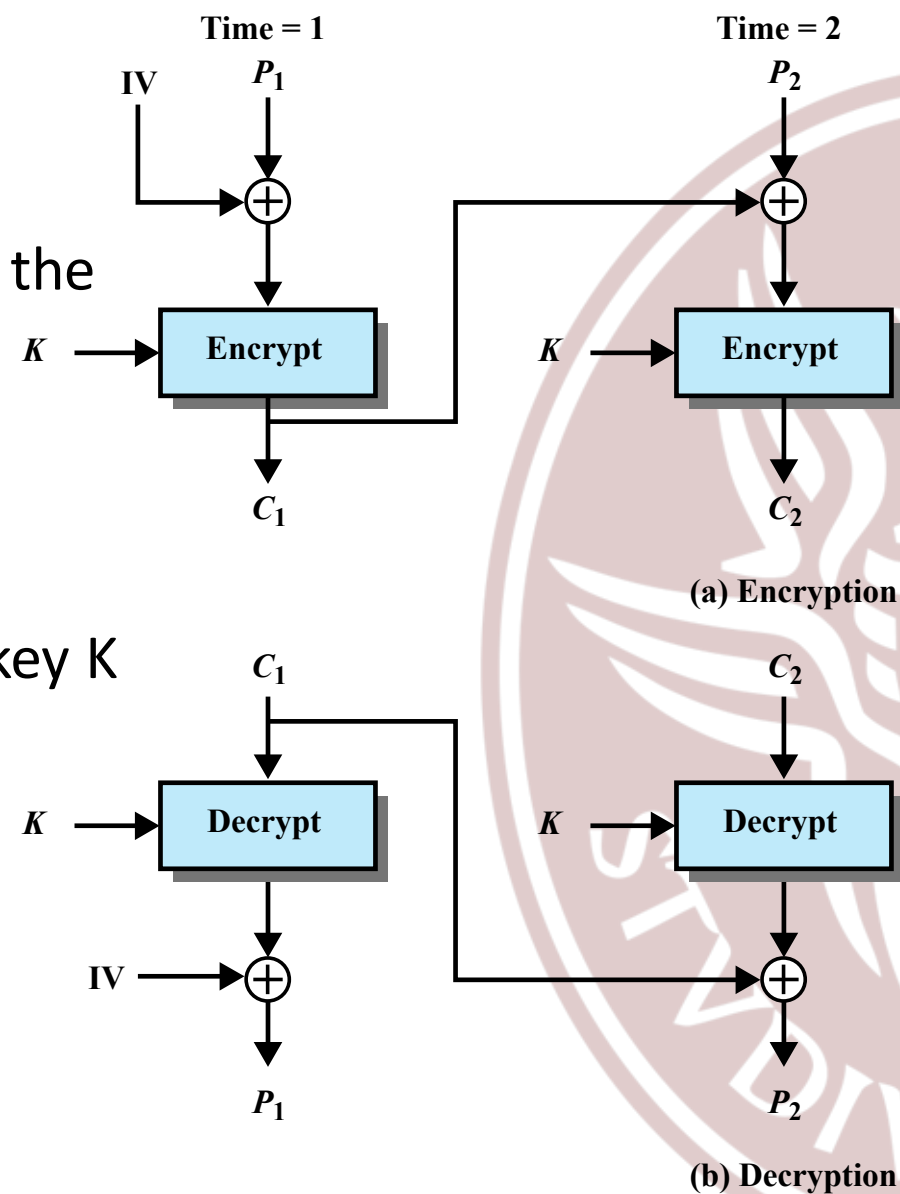
$$C_j = E(K, [C_{j-1} \text{ XOR } P_j])$$

- where $E(K, X)$ is the encryption of plaintext X using key K
Then

$$D(K, C_j) = D(K, E(K, [C_{j-1} \text{ XOR } P_j]))$$

$$D(K, C_j) = C_{j-1} \text{ XOR } P_j$$

$$C_{j-1} \text{ XOR } D(K, C_j) = C_{j-1} \text{ XOR } C_{j-1} \text{ XOR } P_j = P_j$$



Initialization Vector

- To produce the first block of ciphertext, an initialization vector (IV) is XORed with the first block of plaintext.
- On decryption, the IV is XORed with the output of the decryption algorithm to recover the first block of plaintext.
- The IV must be known to both the sender and receiver.
- For maximum security, the IV should be protected as well as the key.
 - This could be done by sending the IV using ECB encryption.
- One reason for protecting the IV is as follows:
 - If an opponent is able to fool the receiver into using a different value for IV, then the opponent is able to invert selected bits in the first block of plaintext.



Error Propagation in CBC

- With the ECB mode, if there is an error in a block of the transmitted ciphertext, only the corresponding plaintext block is affected.
- However, in the CBC mode, this error propagates. For example, an error in the transmitted C_k obviously corrupts P_k and P_{k+1} .
 - Are any blocks beyond P_{k+1} affected?
 - Suppose that there is a bit error in the source version of P_k . Through how many ciphertext blocks is this error propagated? What is the effect at the receiver?

- An error in the transmitted C_k obviously corrupts P_k and P_{k+1} . Are any blocks beyond P_{k+1} affected?
- No. For example, suppose C_k is corrupted. The output block P_{k+2} depends only on the input blocks C_{k+1} and C_{k+2} .
- $C'_k \neq C_k$ corrotto
- $P_k = C_{k-1} \text{ XOR } D(K, C_k)$ corrotto
- $P_{k+1} = C_k \text{ XOR } D(K, C_{k+1})$ corrotto
- $P_{k+2} = C_{k+1} \text{ XOR } D(K, C_{k+2})$ ok

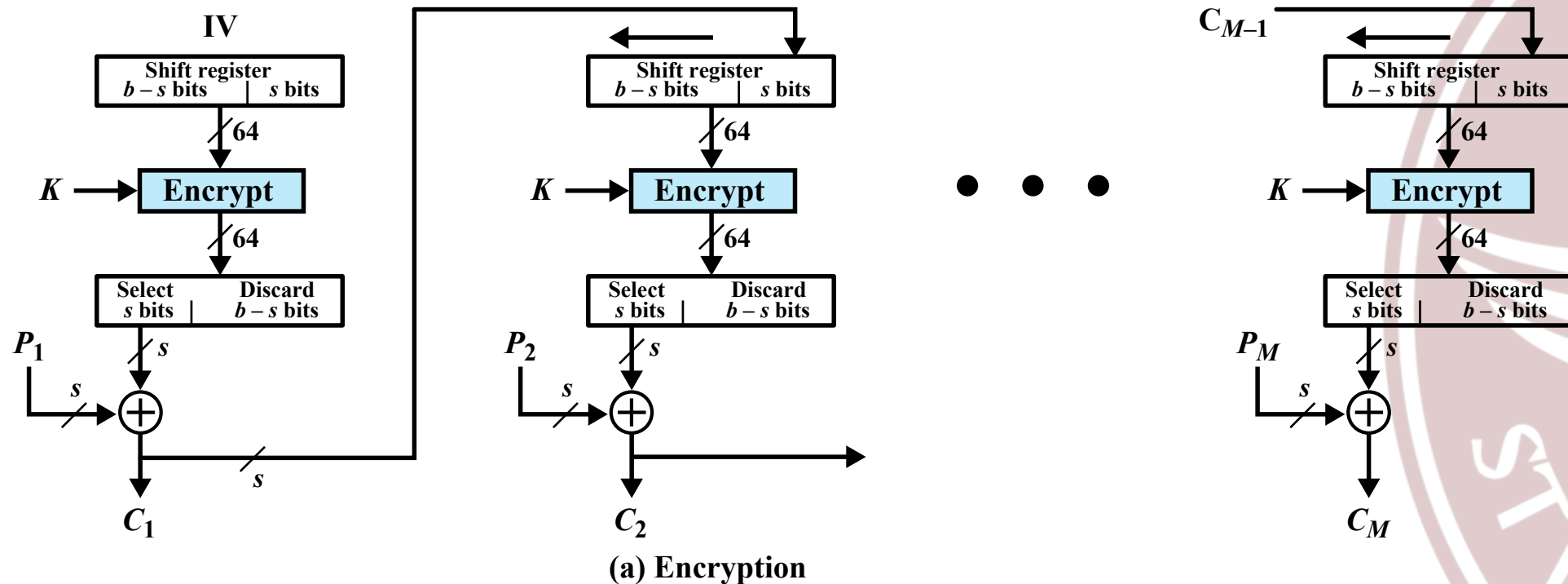


- Suppose that there is a bit error in the source version of P_k . Through how many ciphertext blocks is this error propagated? What is the effect at the receiver?
- An error in P_k affects C_k . But since C_k is input to the calculation of C_{k+1} , C_{k+1} is affected. This effect carries through indefinitely, so that all ciphertext blocks (except the ones before C_k) are affected.
- However, at the receiving end, the decryption algorithm restores the correct plaintext for blocks except the one in error. Therefore, the error only effects the corresponding decrypted plaintext block.

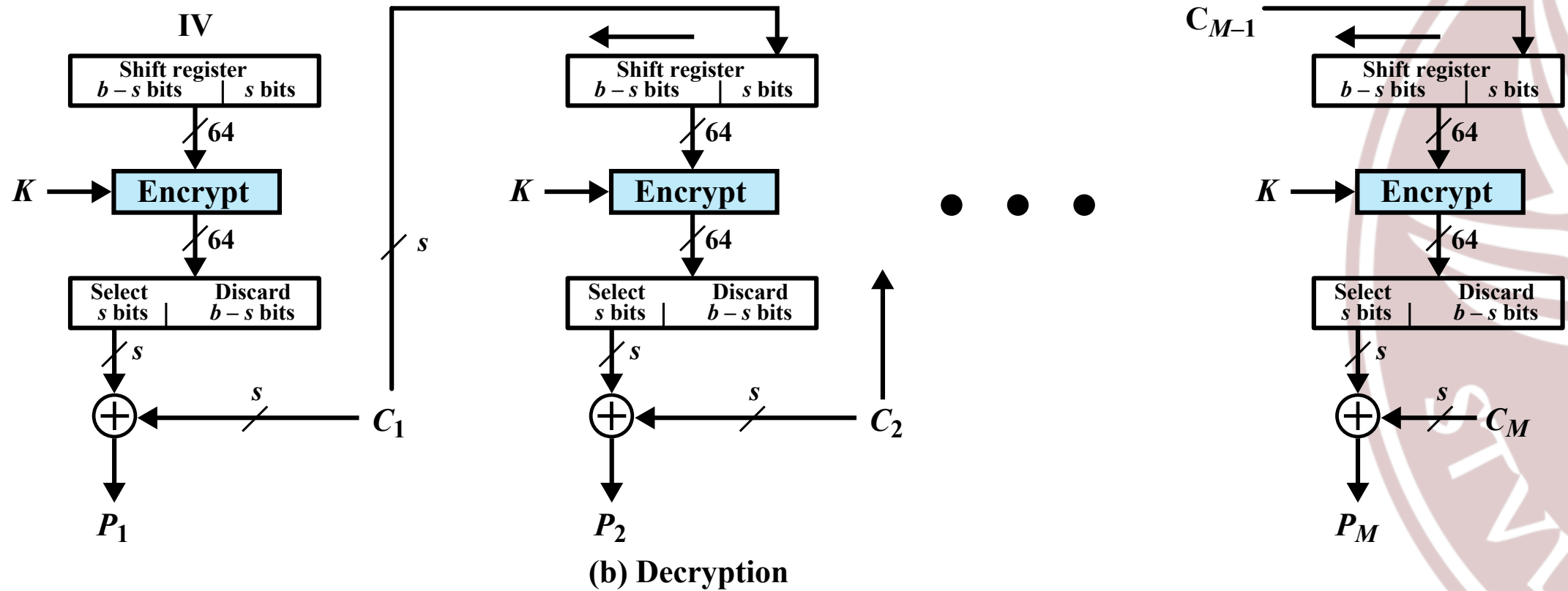
$P'_k \neq P_k$ corrotto
 $C_k = E(K, [C_{k-1} \text{ XOR } P'_k])$ corrotto
 $C_{k+1} = E(K, [C_k \text{ XOR } P_{k+1}])$ corrotto
 $C_{k+2} = E(K, [C_{k+1} \text{ XOR } P_{k+2}])$ corrotto

$P_k = C_{k-1} \text{ XOR } D(K, C_k) = C_{k-1} \text{ XOR } C_{k-1} \text{ XOR } P'_k$ corrotto
 $P_{k+1} = C_k \text{ XOR } D(K, C_{k+1}) = C_k \text{ XOR } C_k \text{ XOR } P_{k+1}$ ok
 $P_{k+2} = C_{k+1} \text{ XOR } D(K, C_{k+2}) = C_{k+1} \text{ XOR } C_{k+1} \text{ XOR } P_{k+2}$ ok

Cipher Feedback (CFB) mode: Converting Block Cipher into Stream Cipher



Cipher Feedback (CFB) mode: Converting Block Cipher into Stream Cipher



Counter (CTR) mode

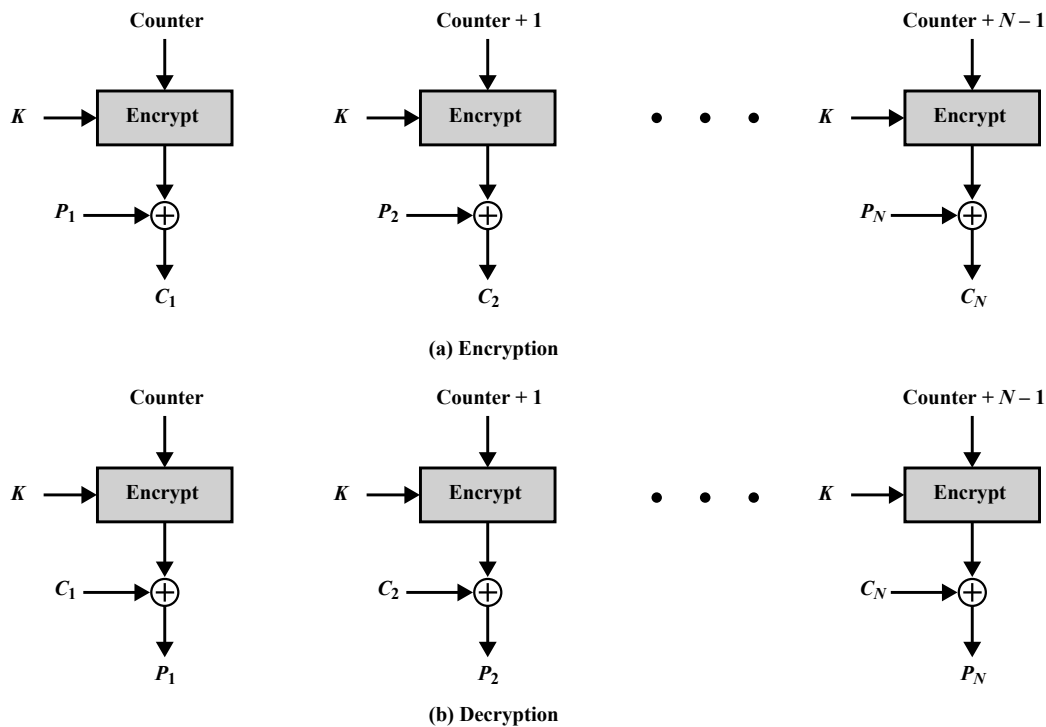


Figure 20.9 Counter (CTR) Mode

- A counter, equal to the plaintext block size is used
- The counter value must be different for each plaintext block that is encrypted (SPS 800-38A)
- Typically, the counter is initialized to some value and then incremented by 1 for each subsequent block (modulo 2^b , where b is the block size).
- Encryption
 - the counter is encrypted and then XORed with the plaintext block to produce the ciphertext block
 - there is no chaining.
- Decryption
 - the same sequence of counter values is used

Advantages of CTR mode

- **Hardware efficiency**
 - CTR mode can be done in parallel on multiple blocks of plaintext or ciphertext; the throughput is only limited by the amount of parallelism that is achieved.
- **Software efficiency**
 - because of the opportunities for parallel execution
- **Preprocessing**
 - The execution of the underlying encryption algorithm does not depend on input of the plaintext or ciphertext.
 - if sufficient memory is available and security is maintained, preprocessing can be used to prepare the output of the encryption boxes that feed into the XOR functions
- **Random access**
 - The i th block of plaintext or ciphertext can be processed in random access fashion
 - Attractive for applications in which a ciphertext is stored and it is desired to decrypt just one block
- **Provable security**
 - It can be shown that CTR is at least as secure as the other modes
- **Simplicity**
 - Unlike ECB and CBC modes, CTR mode requires only the implementation of the encryption algorithm



Cryptanalysis

- The process of attempting to discover the plaintext or key
- The strategy used by the cryptanalyst depends on
 - the nature of the encryption scheme
 - the information available to the cryptanalyst.



Attacks based on the information known by the cryptanalyst

- Ciphertext only
 - Known plaintext
 - Chosen plaintext
 - Chosen ciphertext
 - Chosen text
-
- We assume the opponent always know the encryption algorithm used
 - Condition not always true in the reality



Ciphertext only

- The cryptanalyst know
 - the ciphertext only
- Possible attacks
 - Brute-force; trying all possible keys; impractical if the key space is very large
 - Analysis of the ciphertext itself, generally applying various statistical tests to it
 - the opponent must have some general idea of the type of plaintext that is concealed
 - E.g. English or French text, an EXE file, a Java source listing, an accounting file, and so on.



Known plaintext

- The cryptanalyst know
 - one or more plaintext messages as well as their encryptions (e.g. captured)
 - Or the analyst may know that certain plaintext patterns will appear in a message.
- Attack
 - With this knowledge, the analyst may be able to deduce the key on the basis of the way in which the known plaintext is transformed



Chosen plaintext

- The cryptanalyst
 - is able to get/access the source system
 - to insert into the system a chosen message
- Attack
 - The opponent encrypts a selected message with a patten that can reveal the structure of the key
 - The opponent can access the result of the encryption



Less commonly used

- Chosen ciphertext
 - The opponent has access to the system
 - The opponent select a ciphertext to be dectypted and know the plaintext should be obtained
- Chosen text
 - The opponent has access to the system
 - The opponent can select both the ciphertext and the plaintext and related decrypted/encrypted version



Computationally Secure Encryption Schemes

Encryption is computationally secure if:

- Cost of breaking cipher exceeds value of information
- Time required to break cipher exceeds the useful lifetime of the information

Usually very difficult to estimate the amount of effort required to break

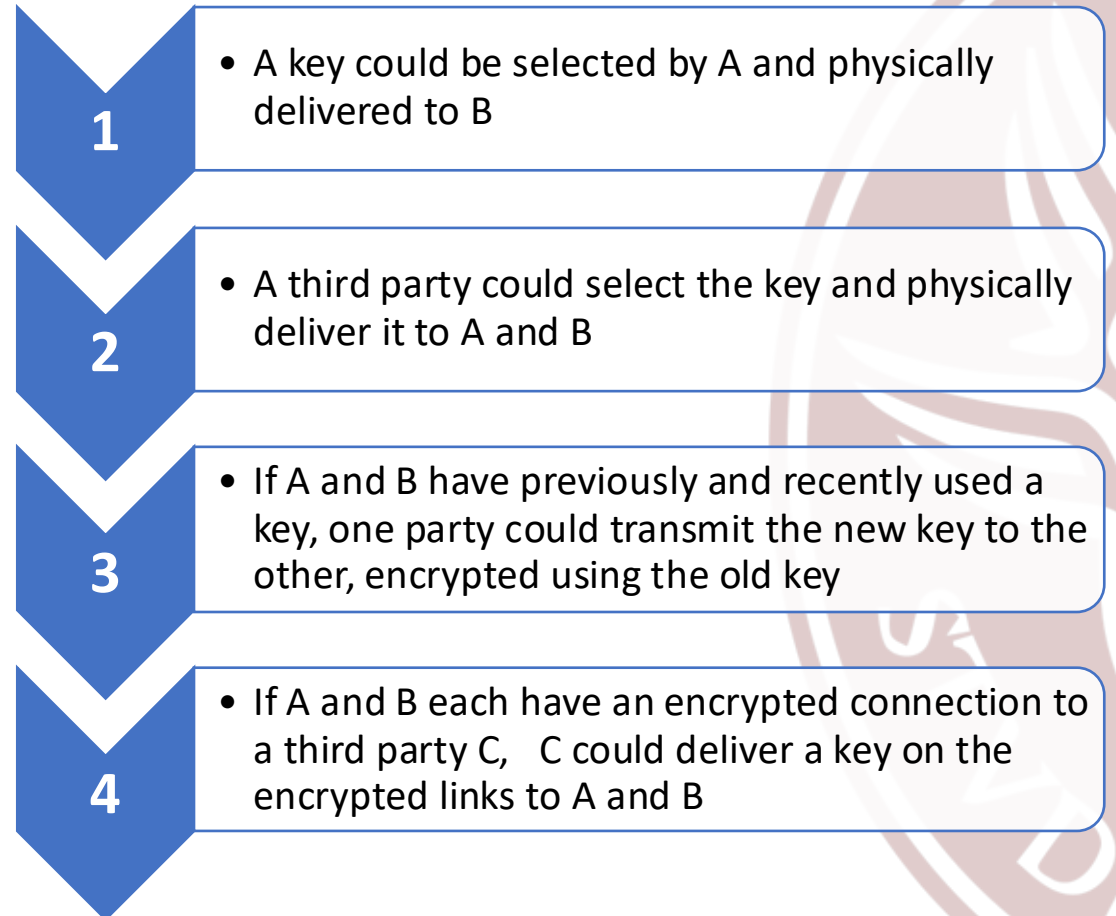
- You can estimate time/cost of a brute-force attack



Key Distribution

- For symmetric encryption to work
 - the two parties to an exchange must share the same
 - that key must be protected from access by others
 - Frequent key changes are usually desirable to limit the amount of data compromised if an attacker learns the key
- The strength of any cryptographic system rests with the key distribution technique
 - The means of delivering a key to two parties that wish to exchange data without allowing others to see the key

Way of Key Distribution



Automatic Key Distribution for Connection-oriented protocols (option 4)

- Two keys
 - Session key: For the duration of a logical connection session, all user data are encrypted with a one-time session key. At the conclusion of the session, or connection, the session key is destroyed.
 - Permanent key: A permanent key is a key used between entities for the purpose of distributing session keys.
- Two entities
 - Key distribution center (KDC)
 - determines which systems are allowed to communicate with each other.
 - provides a one-time session key for that connection.
 - Security service module (SSM):
 - may consist of functionality at one protocol layer
 - performs end-to-end encryption and obtains session keys on behalf of users

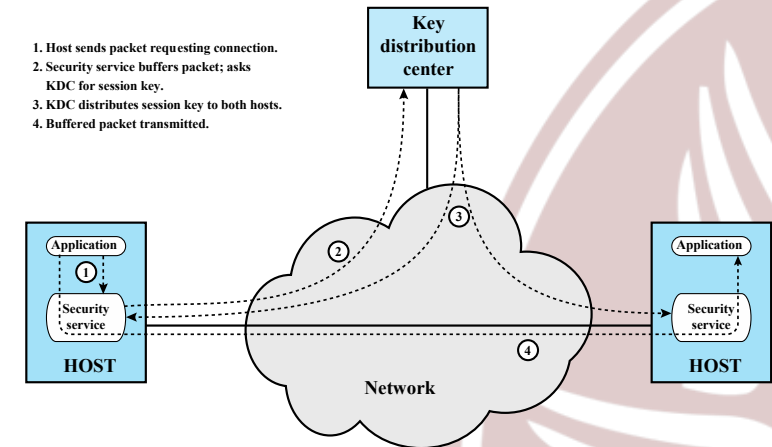
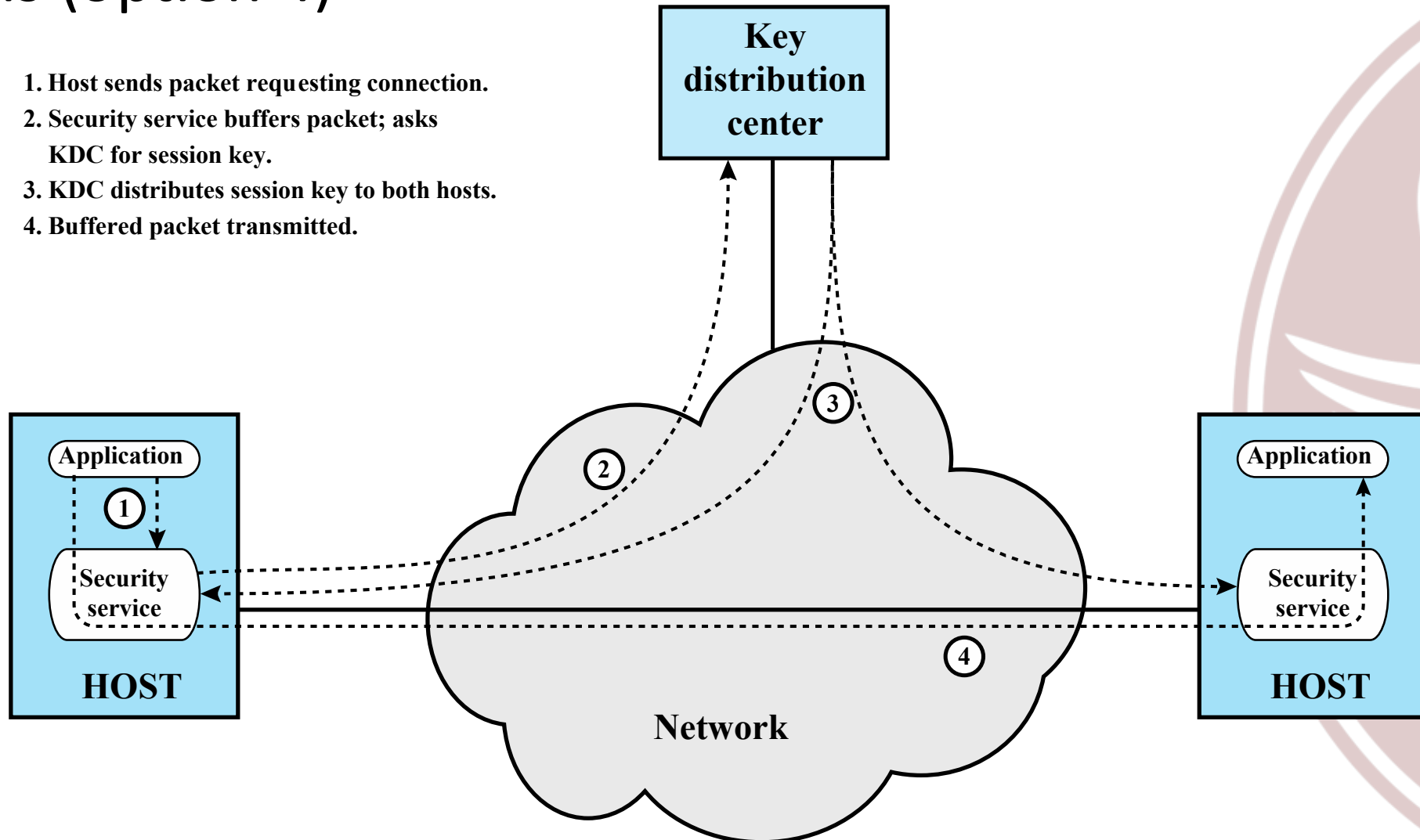


Figure 20.10 Automatic Key Distribution for Connection-Oriented Protocol

Automatic Key Distribution for Connection-oriented protocols (option 4)

1. Host sends packet requesting connection.
2. Security service buffers packet; asks KDC for session key.
3. KDC distributes session key to both hosts.
4. Buffered packet transmitted.



Summary

- Symmetric encryption principles
 - Cryptography
 - Cryptanalysis
 - Feistel cipher structure
- Data encryption standard
 - Data encryption standard
 - Triple DES
- Advanced encryption standard
 - Overview of the algorithm
 - Algorithm details
- Stream ciphers and RC4
 - Stream cipher structure
 - The RC4 algorithm
- Cipher block modes of operation
 - Electronic codebook mode
 - Cipher block chaining mode
 - Cipher feedback mode
 - Counter mode
- Key distribution

